

Práctica de Web Scraping: Extracción de Precios de Gasolineras en Valencia

Alumno/a: Quique Arnandis Escrivà

1. Objetivo de la Práctica

El objetivo de esta práctica ha sido desarrollar un *script* en Python capaz de extraer de forma automatizada los precios actualizados de los distintos tipos de combustible en las gasolineras de la provincia de Valencia. Posteriormente, estos datos no estructurados han sido procesados y almacenados en una base de datos NoSQL (MongoDB) para su futura explotación y análisis.

2. Tecnologías Utilizadas

Para llevar a cabo el proyecto, he configurado el siguiente entorno de trabajo:

- **Lenguaje:** Python 3.
- **Navegación y evasión de bloqueos:** Librería `undetected_chromedriver` (Selenium modificado).
- **Procesamiento de HTML:** Librería `BeautifulSoup4`.
- **Base de Datos:** MongoDB ejecutándose a través de un contenedor de **Docker**, y `pymongo` para la conexión desde Python.

3. Desarrollo y Retos Superados

Durante el desarrollo de la práctica me enfrenté a los sistemas de seguridad de las páginas web modernas.

Inicialmente, intenté realizar la extracción mediante la librería básica `requests`. Sin embargo, la página web objetivo (`elpreciodelagasolina.com`) utiliza el framework **Alpine.js**. Esto significa que el servidor envía un HTML vacío y los datos de los precios se renderizan *a posteriori* utilizando JavaScript. Como `requests` no procesa JavaScript, el resultado de la extracción era nulo (0 datos encontrados).

La Solución: Para solucionar esto, implementé un navegador automatizado utilizando `undetected_chromedriver`. Esta herramienta me permitió:

1. Abrir una sesión de navegador real que es indetectable para los sistemas anti-bots.
2. Esperar el tiempo necesario (renderizado dinámico) para que el JavaScript de la web inyectara los precios en el DOM de la página.
3. Una vez cargados, interceptar el código HTML resultante y pasárselo a `BeautifulSoup` para buscar las etiquetas y clases exactas (`<section>`, `<p`

class="mt-auto">, etc.) que contenían los pares clave-valor (Tipo de combustible - Precio).

Finalmente, el script limpia los datos de texto (usando strip()) y los inserta de forma masiva en la colección precios_valencia de la base de datos energia_spain en MongoDB.

4. Código Fuente

A continuación, se detalla el script principal utilizado para la extracción:

```
gasolina.py X
gasolina.py >...
1 import undetected_chromedriver as uc
2 from bs4 import BeautifulSoup
3 from pymongo import MongoClient
4 import time
5
6 url = "https://www.elpreciodelagasolina.com/gasolineras/valencia"
7
8 print("Iniciando el navegador robot para ejecutar el JavaScript...")
9 opciones = uc.ChromeOptions()
10
11 # AQUÍ ESTÁ LA CORRECCIÓN: Le decimos que use la versión 146 de tu Chrome
12 driver = uc.Chrome(options=opciones, version_main=146)
13
14 try:
15     print(f"Entrando a {url}...")
16     driver.get(url)
17
18     # Le damos 10 segundos de cortesia para que el JavaScript rellene los precios
19     print("X Esperando a que el JavaScript genere los precios en pantalla...")
20     time.sleep(10)
21
22     # Le robamos el HTML ya renderizado con todos los precios visible
23     html_listo = driver.page_source
24     soup = BeautifulSoup(html_listo, "html.parser")
25
26     # 1. Buscamos todas las cajas (secciones) que contienen los precios,
27     # usando exactamente la clase que encontraste en tu captura
28     secciones_precios = soup.find_all("section", class_=lambda c: c and "grid" in c and "gap-2" in c)
29
30     if not secciones_precios:
31         print("X No he encontrado las secciones de precios.")
32     else:
33         print(f"Ha encontrado {len(secciones_precios)} gasolineras! Extrayendo datos...")
34         lista_gasolineras = []
35
36         for seccion in secciones_precios:
37             gasolina = {}
38
39             # 2. Buscar el nombre de la gasolinera
40             # Normalmente el nombre está en un título justo arriba de la sección de precios.
41             # Subimos al contenedor "padre" para buscar un <h2>, <h3> o enlace <a>.
42             padre = seccion.parent
43             titulo = padre.find(["h2", "h3", "h4", "a", "strong"])
44             gasolina["nombre_o_direccion"] = titulo.get_text(strip=True) if titulo else "Gasolinera Desconocida"
45
46             # 3. Extraer los tipos de gasolina y sus precios usando TUS clases
47             tipos = seccion.find_all("p", class_=lambda c: c and "text-brand-secondary" in c)
48             precios = seccion.find_all("p", class_=lambda c: c and "mt-auto" in c)
49
50             # Emparejamos cada tipo (ej. Gasolina 95 E5) con su precio (ej. 1,539 €)
51             for i in range(min(len(tipos), len(precios))):
52                 nombre_tipo = tipos[i].get_text(strip=True)
53                 valor_precio = precios[i].get_text(strip=True)
54                 # lo guardamos en nuestro diccionario
55                 gasolina[nombre_tipo] = valor_precio
56
57             # Si logró extraer algún precio, lo guardamos en la lista final
58             if len(gasolina) > 1:
59                 lista_gasolineras.append(gasolina)
60
61         print(f"\nEXITO TOTAL! Se han extraido {len(lista_gasolineras)} gasolineras con todo detalle.")
62
63         if len(lista_gasolineras) > 0:
64             print("\nMuestra del primer resultado extraido:")
65             print(lista_gasolineras[0])
66             print("\n" * 50)
67
68             # .....
69             # 4. GUARDAR EN MONGODB
70             # .....
71             print("\nGuardando datos en MongoDB...")
72             client = MongoClient("mongodb://localhost:27017/")
73             db = client["energia_spain"]
74             coleccion = db["precios_valencia"]
75
76             coleccion.delete_many({}) # limpiamos la base de datos por si acaso
77             coleccion.insert_many(lista_gasolineras)
78             print(f"Práctica 100% superada y datos guardados en MongoDB!")
79
80 except Exception as e:
81     print(f"X Ocurrió un error: {e}")
82
83 finally:
84     driver.quit()
85     print("Navegador cerrado automáticamente.")
```

5. Demostración de Funcionamiento

Para demostrar la correcta ejecución del código y la persistencia de los datos, presento las siguientes evidencias:

A. Ejecución exitosa en Terminal

El *script* logra evadir los bloqueos, esperar la carga de JavaScript y extraer un total de 198 gasolineras con sus respectivos precios en un formato de diccionario (JSON/BSON).

```
PS C:\Users\larran\OneDrive\Desktop\IA_1_BigData\Sistemas_de_Big_Data\Gasolina> python gasolina.py
Iniciando el navegador robot para ejecutar el JavaScript...
Entrando a https://www.elpreciodelagasolina.com/gasolineras/valencia...
Esperando a que el JavaScript genere los precios en pantalla...
¡He encontrado 198 gasolineras! Extrayendo datos...

¡ÉXITO TOTAL! Se han extraído 198 gasolineras con todo detalle.

Muestra del primer resultado extraído:
{'nombre_o_direccion': 'precio', '': '', 'Gasolina 95 E5': '1,419\vae8', 'Gasóleo A habitual': '1,586\vae8', 'Gasolina 98 E5': '1,546\vae8', 'Gasóleo Premium': '1,649\vae8', 'Gasolina 95 E5 Premium': '1,469\vae8', 'GLP': '0,726\vae8', 'Adblue': '0,68\vae8', 'Diésel Renovable': '1,889\vae8', 'Gas natural (GNC)': '1,313\vae8', 'Gasolina Renovable': '1,685\vae8', 'Gasóleo B': '1,169\vae8'}
-----
Guardando datos en MongoDB...
¡Práctica 1888 superada y datos guardados en MongoDB!
Navegador cerrado automáticamente.
Exception ignored in: <function Chrome.__del__ at 0x0000015C9D067EC0>
Traceback (most recent call last):
  File "C:\Users\larran\AppData\Local\Programs\Python\Python313\Lib\site-packages\undetected_chromedriver\__init__.py", line 843, in __del__
  File "C:\Users\larran\AppData\Local\Programs\Python\Python313\Lib\site-packages\undetected_chromedriver\__init__.py", line 798, in quit
OSError: [WinError 6] Controlador no válido
```

B. Almacenamiento en Base de Datos (MongoDB)

Comprobación visual de que los datos han viajado correctamente desde el script de Python hasta el contenedor Docker de MongoDB, estructurados y listos para Big Data.

